

## 1. Client-Server-Architektur

- Serverprozess mit dauerhaft offenem TCP-Socket
- Verarbeitung von Befehlen (SEND, LIST, READ, DEL)
- Client als Terminalprogramm, das Befehle an den Server sendet
- Dateibasierte Speicherung im „spool“-Verzeichnis, getrennt nach Benutzern
- Server zustandslos hinsichtlich Verbindungen, aber persistent in der Mailablage

### Kurzbeschreibung:

Der Server übernimmt sämtliche Logik und Datenhaltung, während der Client nur der Interaktion und Protokollübertragung dient. Jede Mail wird als Datei im Benutzerordner gespeichert, sodass kein Datenbanksystem notwendig ist.

## 2. Verwendete Technologien

- Implementierung in **C++17**
- **TCP-Sockets** (send(), recv()) für Netzwerkkommunikation
- **std::filesystem** für Ordner- und Dateiverwaltung
- Linux-Systemfunktionen (unistd.h, errno, fork())
- Eigenes Textprotokoll anstelle von SMTP/IMAP
- Kompilation über einfaches **Makefile** (Targets: client, server, clean)

## 3. Entwicklungsstrategie & erforderliche Anpassungen

- Entwicklung in mehreren Schritten: Socket-Setup → Befehlsparser → MailStore → Client-Anbindung
- Refactoring in getrennte Klassen TWMailerServer und TWMailerClient zur besseren Wartbarkeit
- Einführung robuster Übertragungsfunktionen zur Vermeidung von Paketverlust durch TCP-Fragmentierung
- Umstellung auf std::filesystem, um C++17-konform zu bleiben
- Anpassung des Makefiles für portable Builds
- Erweiterung um Fehlerbehandlung (ungültige Befehle, defekte Spool-Verzeichnisse, IO-Fehler)